17. Letter Combinations of a Phone Number Medium

Acceptance: 65.3% | LeetCode

Given a string containing digits from `2-9` inclusive, return all possible letter combinations that the number could represent. Return the answer in **any order**.

A mapping of digits to letters (just like on the telephone buttons) is given below. Note that 1 does not map to any letters.

Example 1:

```
**Input:** digits = "23"
**Output:**
["ad","ae","af","bd","be","bf","cd","ce","cf"]
```

Example 2:

```
**Input:** digits = "2"
**Output:** ["a","b","c"]
```

Constraints:

- `1 <= digits.length <= 4`
- `digits[i]` is a digit in the range `['2', '9']`.

Claude Opus Solution

Letter Combinations of a Phone Number

1. Intuition

The key insight is that this is a **Cartesian product** problem. Each digit independently maps to a set of letters, and we need every possible combination where we pick **one letter from each digit's set**.

Think of it like a tree: - Each level of the tree corresponds to one digit. - Each node branches into the letters that digit maps to. - Every **root-to-leaf path** forms one valid combination.

```
       "23"
        / | \
       a  b  c        <- choices for digit '2'
      /|\ /|\ /|\
     d e f ...         <- choices for digit '3'
```

This screams **backtracking (DFS)**: build combinations incrementally, and when we've chosen a letter for every digit, record the result.

---

2. Approach (Pseudocode)

```
1. If digits is empty, return [] (edge case).
2. Build a map: digit -> string of letters.
3. Define dfs(index, path):
      a. BASE CASE: if index == len(digits):
            - path is complete, add joined path to results
            - return
      b. For each letter mapped to digits[index]:
            - choose: append letter to path
            - explore: dfs(index + 1, path)
            - un-choose (backtrack): remove letter from path
4. Start dfs(0, "") and return results.
```

The **choose** → **explore** → **un-choose** pattern is the heart of backtracking.

---

3. Python Solution

```python
from typing import List


class Solution:
    def letterCombinations(self, digits: str) -> List[str]:
        # Edge case: empty input yields no combinations.
        if not digits:
            return []
```

```
        # Map each digit to its corresponding letters (telephone keypad).
        digit_to_letters = {
            "2": "abc", "3": "def", "4": "ghi", "5": "jkl",
            "6": "mno", "7": "pqrs", "8": "tuv", "9": "wxyz"
        }

        result = []          # Stores all completed combinations.
        path = []            # Current combination being built (used as a stack).

        def dfs(index: int) -> None:
            # BASE CASE: we've picked a letter for every digit.
            if index == len(digits):
                # path now holds one full combination -> save it.
                result.append("".join(path))
                return

            # Get all letters for the current digit.
            letters = digit_to_letters[digits[index]]

            # Try each letter as the choice for this position.
            for letter in letters:
                path.append(letter)   # CHOOSE: commit to this letter
                dfs(index + 1)        # EXPLORE: solve the rest of the digits
                path.pop()            # UN-CHOOSE: backtrack to try next letter

        # Kick off recursion from the first digit (index 0).
        dfs(0)
        return result
```

---

4. Complexity Analysis

Let `n` = length of `digits`.

Time Complexity: `O(4^n · n)` - Each digit maps to at most **4 letters** (e.g., '7' → "pqrs", '9' → "wxyz"). - With `n` digits, there are up to `4^n` total combinations (leaves in the tree). - Building each final string via `"".join(path)` costs `O(n)`. - Therefore: `O(4^n · n)`.

Space Complexity: `O(n)` (excluding the output) - The recursion depth is at most `n` (one frame per digit). - The `path` list holds at most `n` characters at any time. - If we **include** the output, it's `O(4^n · n)` to store all combinations.

---

5. Alternative Approaches

A) Iterative BFS / Queue-based building

Build combinations level by level instead of using recursion.

```
def letterCombinations(self, digits: str) -> List[str]:
    if not digits:
        return []
    digit_to_letters = {
        "2": "abc", "3": "def", "4": "ghi", "5": "jkl",
        "6": "mno", "7": "pqrs", "8": "tuv", "9": "wxyz"
    }
    # Start with one empty prefix, then expand for each digit.
    result = [""]
    for digit in digits:
        # Combine every existing prefix with every letter of this digit.
        result = [prefix + letter
                  for prefix in result
                  for letter in digit_to_letters[digit]]
    return result
```

- Same `O(4^n · n)` time, clean and Pythonic.
- Avoids explicit recursion stack but holds intermediate results in memory.

B) `itertools.product`

Directly compute the Cartesian product.

```
from itertools import product
```

```python
def letterCombinations(self, digits: str) -> List[str]:
    if not digits:
        return []
    mapping = {
        "2": "abc", "3": "def", "4": "ghi", "5": "jkl",
        "6": "mno", "7": "pqrs", "8": "tuv", "9": "wxyz"
    }
    # product expands the Cartesian product of each digit's letter set.
    groups = [mapping[d] for d in digits]
    return ["".join(combo) for combo in product(*groups)]
```

- Most concise; leverages the built-in that's purpose-built for this exact pattern.

Key Takeaway

This problem is a classic **backtracking / Cartesian product** template. Whenever you must enumerate **all combinations across independent choice sets**, the `choose → explore → un-choose` pattern (or equivalently `itertools.product`) is the go-to approach. Recognizing the tree structure is the unlock.

Codex Solution

1. Intuition

Each digit maps to a small set of possible letters:

```
2 -> abc
3 -> def
...
7 -> pqrs
9 -> wxyz
```

For every digit, we need to choose exactly one letter from its mapped letters.

So for input:

```
"23"
```

We choose:

```
one letter from "abc"
one letter from "def"
```

This is a classic **backtracking / DFS / Cartesian product** problem.

We build combinations one character at a time:

```
choose letter for digit 0
choose letter for digit 1
choose letter for digit 2
...
```

When the current combination has the same length as `digits`, we add it to the answer.

---

2. Approach

Backtracking idea

Maintain:

- `index`: which digit we are currently processing
- `path`: the current partial combination
- `result`: all complete combinations

Pseudocode

```
letterCombinations(digits):
    if digits is empty:
        return []

    create digit_to_letters mapping
```

```
    result = []
    path = []

    backtrack(index):
        if index == length of digits:
            add joined path to result
            return

        current_digit = digits[index]
        possible_letters = digit_to_letters[current_digit]

        for each letter in possible_letters:
            add letter to path
            backtrack(index + 1)
            remove letter from path

    backtrack(0)

    return result
```

---

3. Python Solution

```python
from typing import List


class Solution:
    def letterCombinations(self, digits: str) -> List[str]:
        # Edge case:
        # If there are no digits, there are no combinations.
        if not digits:
            return []

        # Mapping from phone digits to their corresponding letters.
        digit_to_letters = {
            "2": "abc",
            "3": "def",
            "4": "ghi",
            "5": "jkl",
            "6": "mno",
            "7": "pqrs",
            "8": "tuv",
            "9": "wxyz"
        }

        result = []  # Stores all complete letter combinations
        path = []    # Stores the current combination being built

        def backtrack(index: int) -> None:
            """
            index represents the current position in the digits string.

            For example, if digits = "23":
            - index = 0 means we are choosing a letter for digit '2'
            - index = 1 means we are choosing a letter for digit '3'
            """

            # Base case:
            # If we have chosen one letter for every digit,
            # then path is a complete combination.
            if index == len(digits):
                result.append("".join(path))
                return

            # Get the current digit we need to process.
            current_digit = digits[index]

            # Get all possible letters for this digit.
            possible_letters = digit_to_letters[current_digit]

            # Try every possible letter for the current digit.
            for letter in possible_letters:
```

```
                # Choose this letter.
                path.append(letter)

                # Move on to the next digit.
                backtrack(index + 1)

                # Undo the choice so we can try the next letter.
                path.pop()

        # Start building combinations from the first digit.
        backtrack(0)

        return result
```

---

4. Complexity Analysis

Let:

- `n` be the length of `digits`
- Each digit maps to at most `4` letters

Digits `7` and `9` map to `4` letters, while the others map to `3`.

Time Complexity

Worst case:

```
O(n * 4^n)
```

Why?

- There can be up to `4^n` combinations.
- Each complete combination has length `n`.
- Joining `path` into a string costs `O(n)`.

So total time is:

```
O(n * 4^n)
```

More precisely, if each digit `i` maps to `k_i` letters, then the number of combinations is:

```
k_1 * k_2 * ... * k_n
```

So the more exact time complexity is:

```
O(n * product(k_i))
```

Space Complexity

If we ignore the output list:

```
O(n)
```

Because:

- The recursion depth is at most `n`
- The `path` list stores at most `n` characters

If we include the output list:

```
O(n * 4^n)
```

Because we may store up to `4^n` strings, each of length `n`.

---

5. Alternative Approaches

1. Iterative BFS-style construction

Start with:

```
[""]
```

For each digit, append each possible letter to all existing combinations.

Example:

```
digits = "23"

start: [""]
after '2': ["a", "b", "c"]
after '3': ["ad", "ae", "af", "bd", "be", "bf", "cd", "ce", "cf"]
```

This avoids recursion but has the same complexity.

---

2. Using `itertools.product`

Python has a built-in function for Cartesian products:

```
from itertools import product
```

You can convert each digit to its letter string and compute the Cartesian product.

Example:

```
from typing import List
from itertools import product


class Solution:
    def letterCombinations(self, digits: str) -> List[str]:
        if not digits:
            return []

        digit_to_letters = {
            "2": "abc",
            "3": "def",
            "4": "ghi",
            "5": "jkl",
            "6": "mno",
            "7": "pqrs",
            "8": "tuv",
            "9": "wxyz"
        }

        groups = [digit_to_letters[digit] for digit in digits]

        return ["".join(combo) for combo in product(*groups)]
```

This is concise, but backtracking is usually preferred in interviews because it clearly demonstrates the recursive decision-making process.